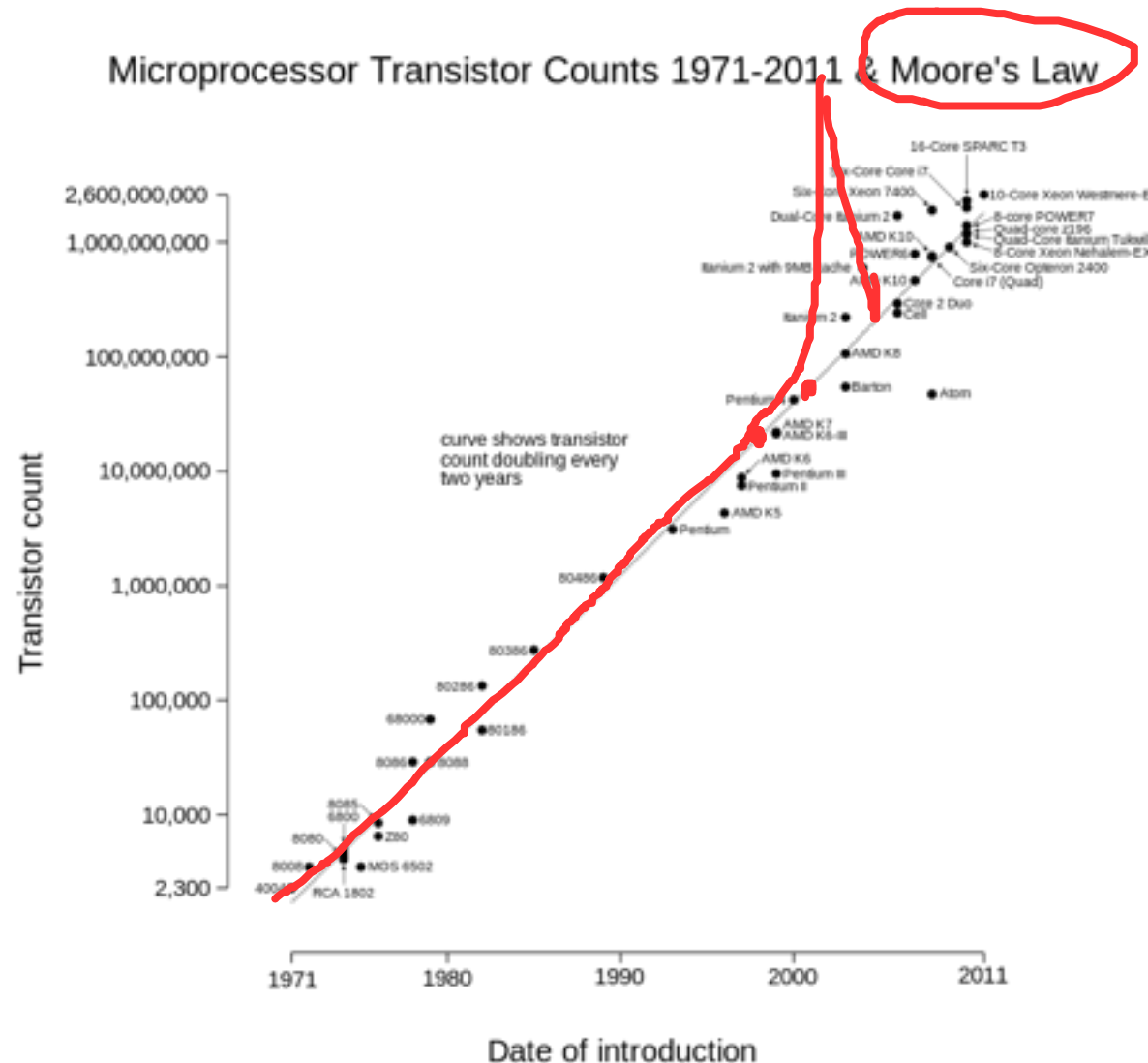


- Machine learning taxonomy.
 - Supervised learning.
 - Classification.
- Imbalanced data.
 - Random over/under sampling.
 - SMOTE.
 - Cost-sensitive classification.
- Semi-supervised learning.
 - Self-learning.
 - Multi-view learning.
 - Co-Training.
- Stacked generalization.
- Data representations.
- Multi-user evaluation.
- Baseline classifiers.

Given that the computational power of machines has been increasing in the last years, it would be desirable to use that power to process that huge amounts of data to extract knowledge from it.



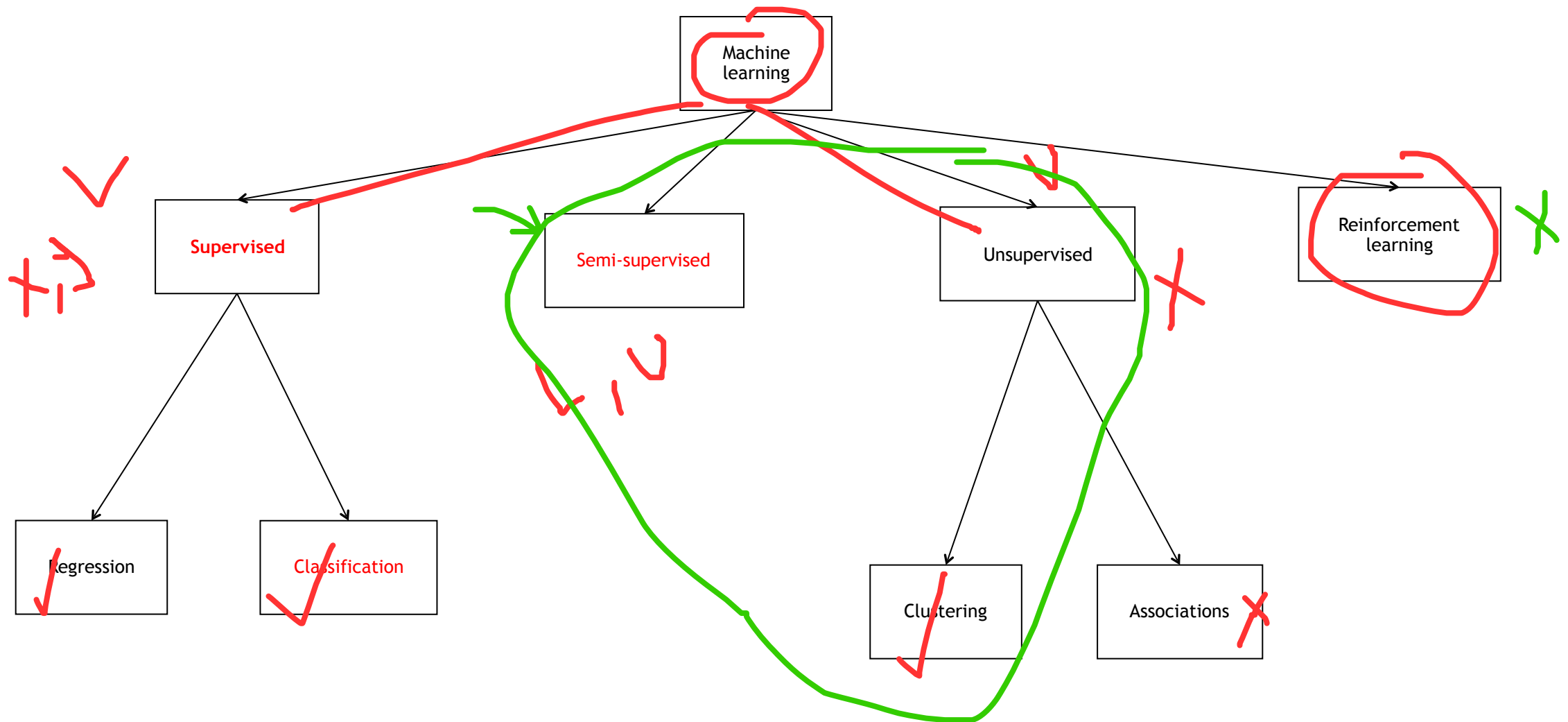
Machine learning can be thought of (but not limited to), as a set of computational algorithms that *automatically* find interesting patterns and relationships from data.

✓ *“The basic principle of machine learning is the automatic modeling of underlying processes that have generated the collected data.” (I. Kononenko and M. Kukar, 2007).*

Learning

- (I. Kononenko and M. Kukar., 2007) defined *learning* as “any modification of the system that improves its performance in some problem solving task.”
- The result of learning is knowledge that the system can use to solve new problems. An algorithm infers the properties of a given set of data and that information allows it to make predictions about other data that it might see in the future.
- This is possible because almost all nonrandom data contains patterns which allows a machine to generalize (T. Segaran, 2007).

ML taxonomy



✓ Imbalanced data

- Random over/under sampling
- SMOTE
- Cost sensitive classification



You trained a model to **predict cancer** from image data using a state of the art Hierarchical siamese CNN with dynamic kernel activations...

Your model has an accuracy of 99.9%



But... WTH!?

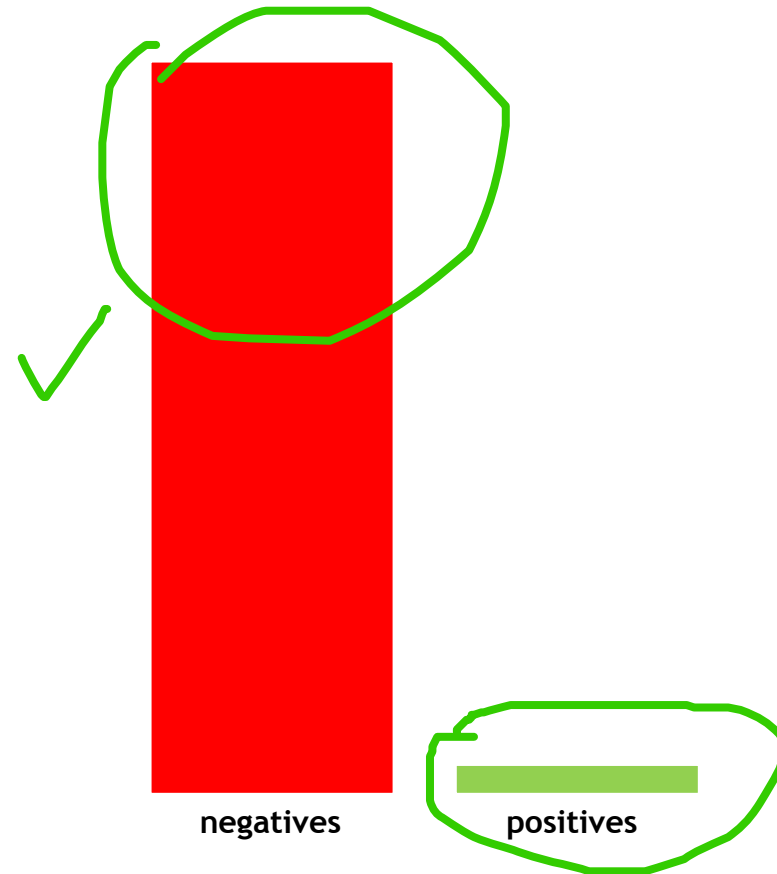


By looking at the confusion matrix you realize that the model does not detect any of the positive examples.



100	10
0	0

After plotting your class distribution you see that you have thousands of negative examples but just a couple of positives.



Classifiers try to reduce the overall error so they can be biased towards the majority class.

Negatives = 998

Positives = 2

By always predicting a negative class the accuracy will be 99.8% !!

Your dataset is imbalanced.

Now what??



✓ What can you do?

- Collect more data (difficult in many domains)

- ✓ Delete data from the majority class J

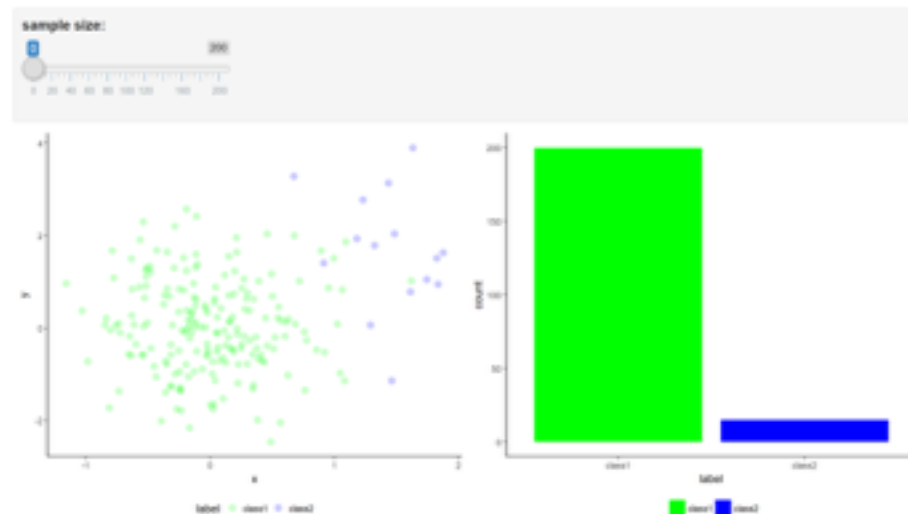
- ✓ Create synthetic data D

- ✓ Adapt your learning algorithm (cost sensitive classification) C

Random over/under sampling

- ✓ • **Random oversampling**: randomly duplicate data points from the minority class.
- ✓ • **Random undersampling**: randomly delete data points from the majority class.

Demo

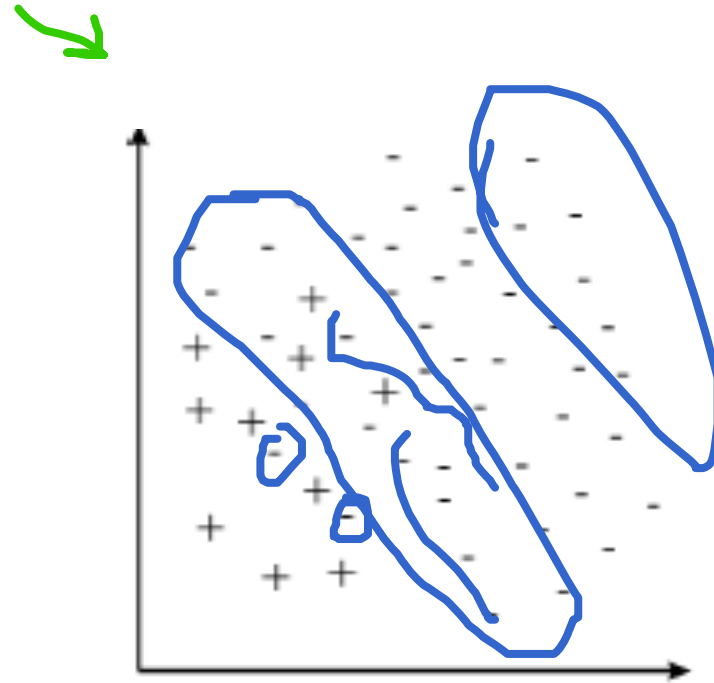


Problems with these approaches:

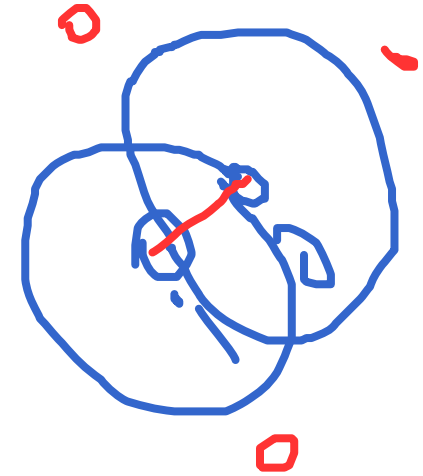
- ✓ Loss of information (in the case of under sampling)
- ✓ Overfitting and fixed boundaries (over sampling)

Four Groups of Negative Examples

- Noise examples
- Borderline examples
 - Borderline examples are unsafe since a small amount of noise can make them fall on the wrong side of the decision border.
- Redundant examples
- Safe examples



Tomek links [1]

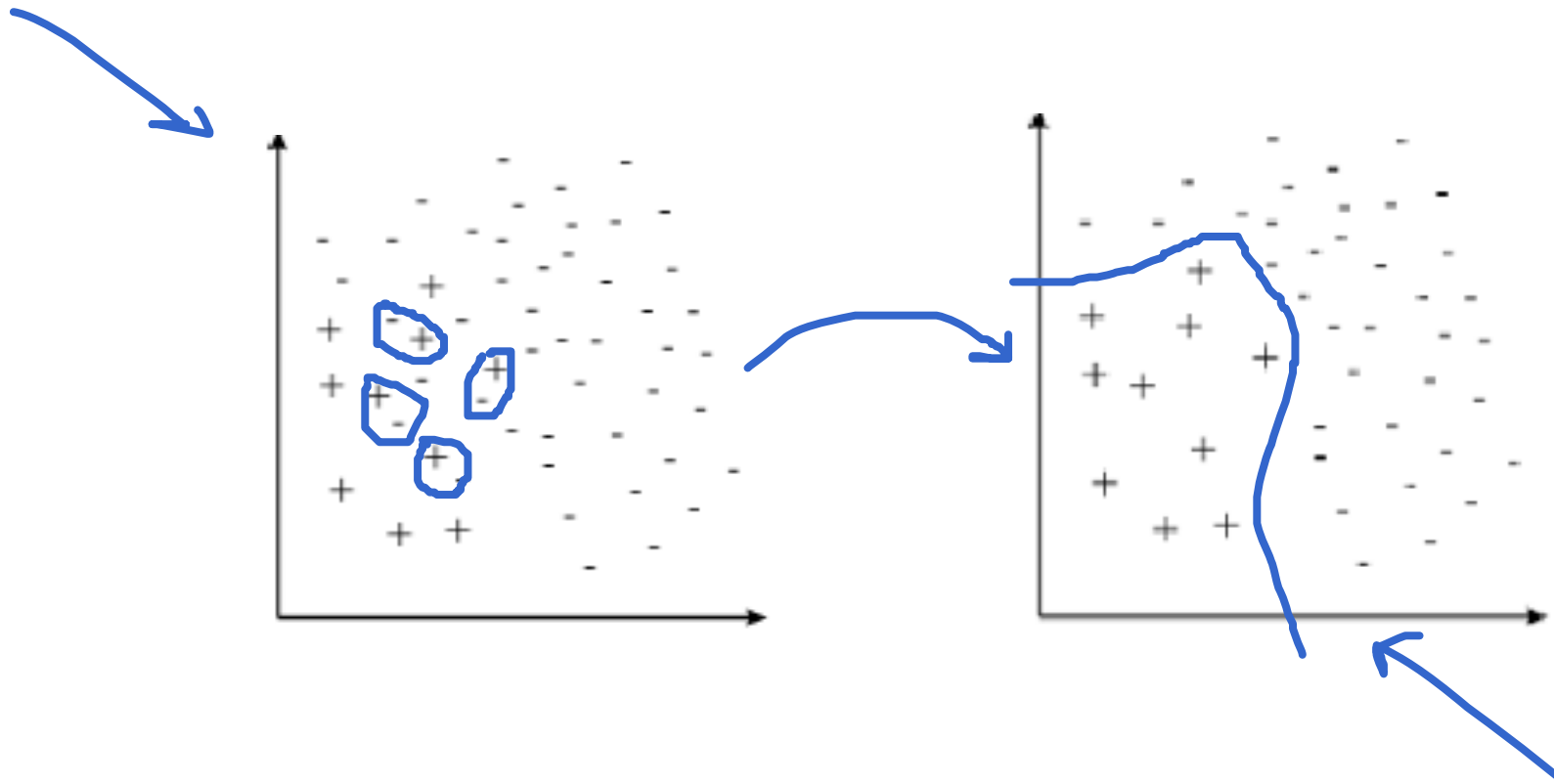


- To remove both noise and borderline examples

- Tomek link

- E_i, E_j belong to different classes, $d(E_i, E_j)$ is the distance between them.
- A (E_i, E_j) pair is called a Tomek link if there is no example E_l , such that $d(E_i, E_l) < d(E_i, E_j)$ or $d(E_j, E_l) < d(E_i, E_j)$.

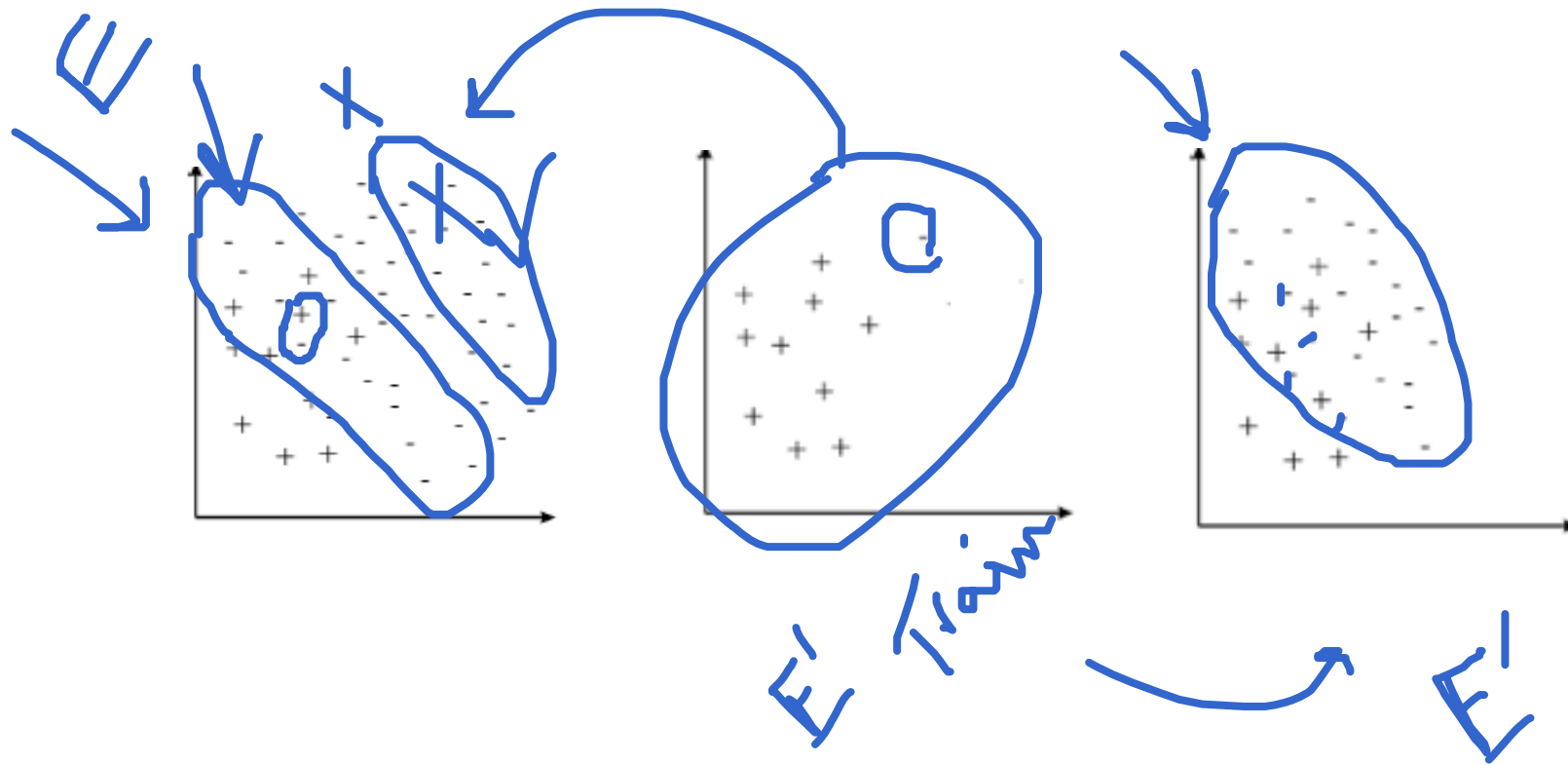
Tomek links



Condensed Nearest Neighbor Rule (CNN rule) [2]

- To pick out points near the boundary between the classes
- To find a consistent subset of examples.
 - A subset $E' \subseteq E$ is consistent with E if using a 1-nearest neighbor, E' correctly classifies the examples in E
- Algorithm:
 - Let E be the original training set
 - Let E' contains all positive examples from S and one randomly selected negative example
 - Classify E with the 1-NN rule using the examples in E'
 - Move all misclassified example from E to E'
- Sensitive to noise. Noisy examples are likely to be misclassified, many of them will be added to the training set.

Condensed Nearest Neighbor Rule (CNN rule)

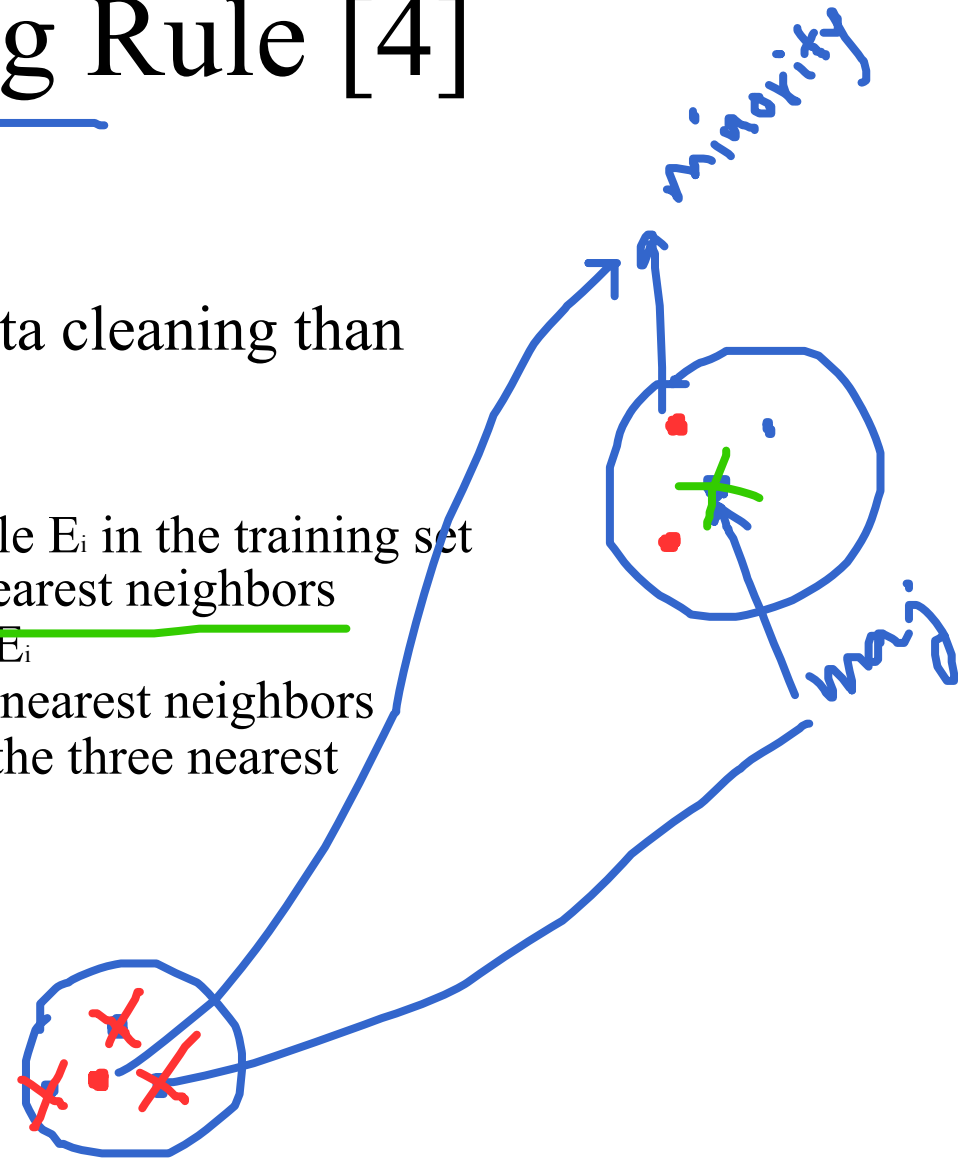


✓ One-sided selection [3] vs CNN+Tomek links

- One-sided selection
 - Tomek links + CNN
- CNN + Tomek links
 - Proposed by the author
 - ✓ – Finding Tomek links is computationally demanding, it would be computationally cheaper if it was performed on a reduced data set.

✓ Neighborhood Cleaning Rule [4]

- To remove majority class examples
- Different from OSS, emphasize more data cleaning than data reduction
- Algorithm:
 - Find three nearest neighbors for each example E_i in the training set
 - If E_i belongs to majority class, & the three nearest neighbors classify it to be minority class, then remove E_i
 - If E_i belongs to minority class, and the three nearest neighbors classify it to be majority class, then remove the three nearest neighbors



References

- ✓ [1] Tomek, I. Two Modifications of CNN. *IEEE Transactions on Systems Man and Communications SMC-6* (1976), pp. 769-772
- ✓ [2] Hart, P. E. The Condensed Nearest Neighbor Rule. *IEEE Transactions on Information Theory IT-14* (1968), 515-516
- [3] Kubat, M., and Matwin, S. Addressing the Course of Imbalanced Training Sets: One-sided Selection. In *ICML* (1997), pp. 179-186.
- [4] Laurikkala, J. Improving Identification of Difficult Small Classes by Balancing Class Distribution. Tech. Rep. A-2001-2, University of Tampere, 2001.
- ✓ [5] Wilson, D. L. Asymptotic Properties of Nearest Neighbor Rules Using Edited Data. *IEEE Transactions on Systems, Man, and Communications* 2, 3 (1972), 408-421.
- ✓ [6] Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. SMOTE: Synthetic Minority Over-sampling Technique. *JAIR* 16 (2002), 324-357

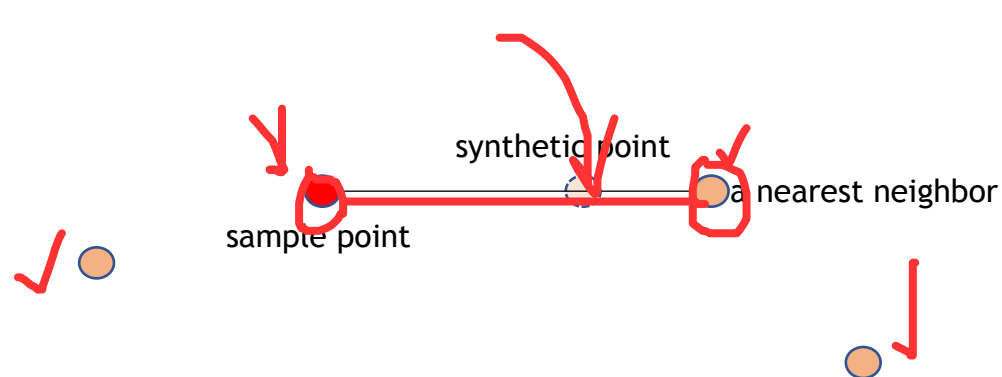
SMOTE

- Synthetic Minority Over-sampling Technique (Chawla).
- Creates new data points from the minority class.
- Operates in the feature space.

- Main steps:

1. Take the difference between a sample point and one of its nearest neighbors.
2. Multiply the difference by a random number between 0 and 1 and add it to the feature vector.

This causes the selection of a random point along the line segment between two specific features.

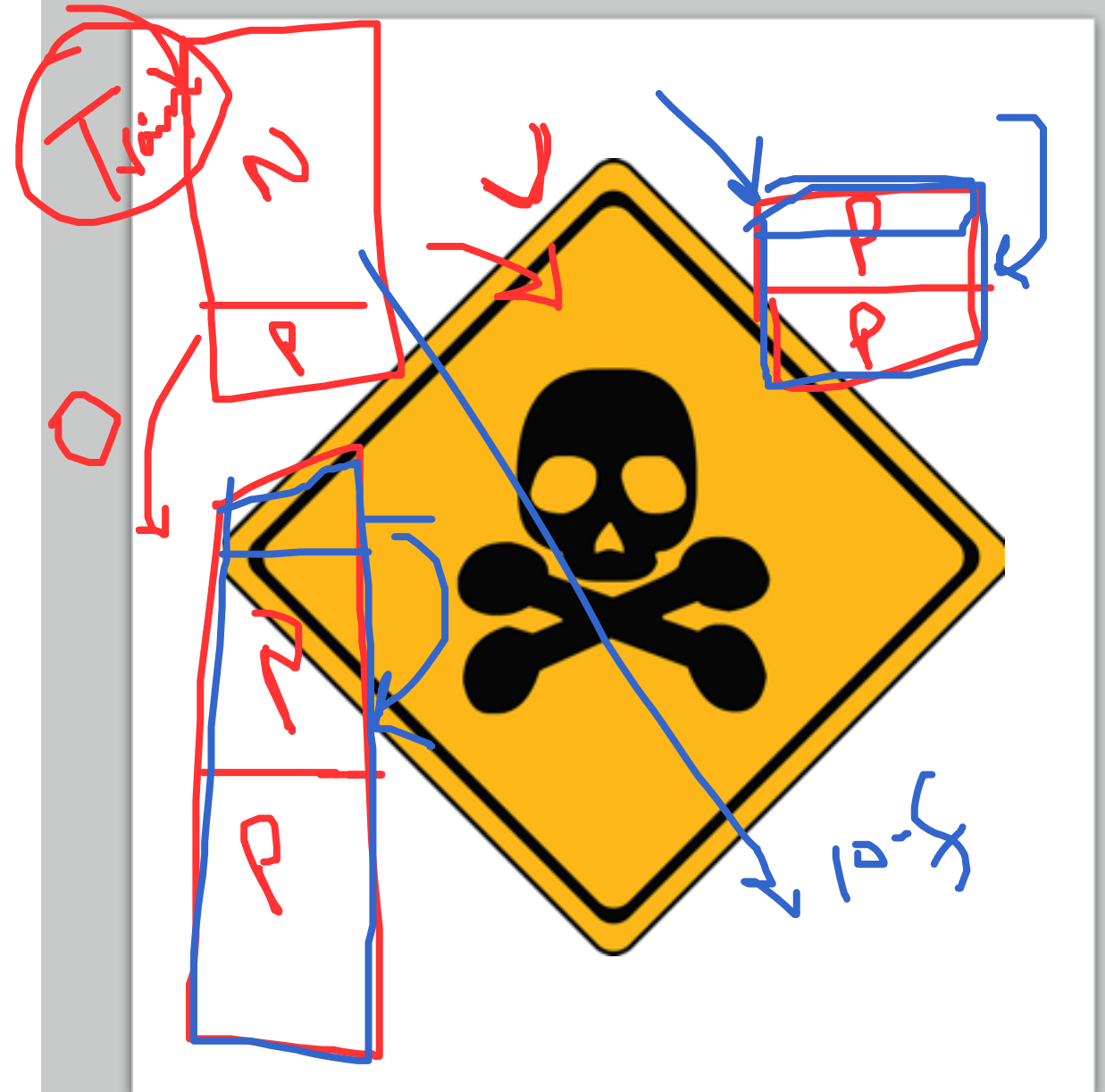


A diagram illustrating the calculation of a synthetic point's coordinates. It shows a horizontal line segment with a red dot in the middle. To the left of the red dot is a red circle containing the coordinates $(2, 1)$. To the right of the red dot is a red circle containing the coordinates $(4, 5)$. Below the line segment is a blue equation:
$$\left[2 + \cdot 4(4 - 2) \quad 1 + \cdot 4(5 - 1) \right]$$

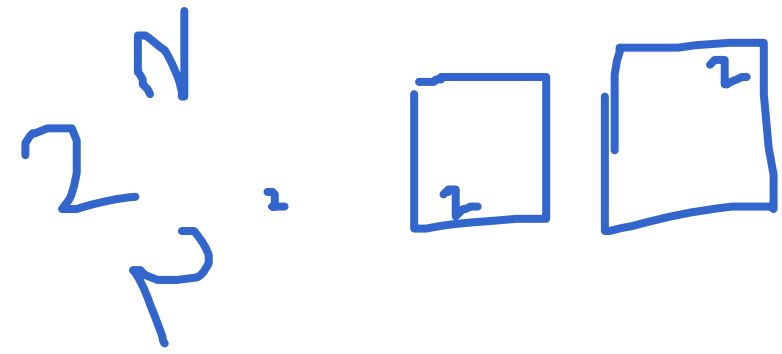
Danger of information injection and overfitting

Do not create synthetic points on the entire dataset before splitting into train/test sets.

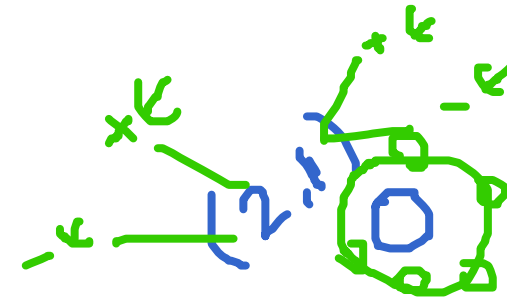
- ✓ Perform the preprocessing just on the training data!!
- For k-fold cross validation, you have to do it for each fold (just on the training set).



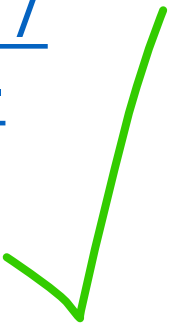
✓ For images:



- Augment training data by applying image transformations: rotate, scale, shift, etc.



- Keras provides functionalities for data augmentation: <https://blog.keras.io/building-powerful-image-classification-models-using-very-little-data.html>



Type I error
(false positive)

Type II error
(false negative)



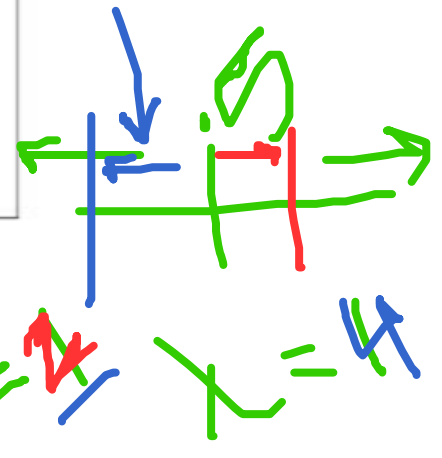
TP: true positives

FP: false positives

0	λ
μ	0

FN: false negatives

TN: true negatives



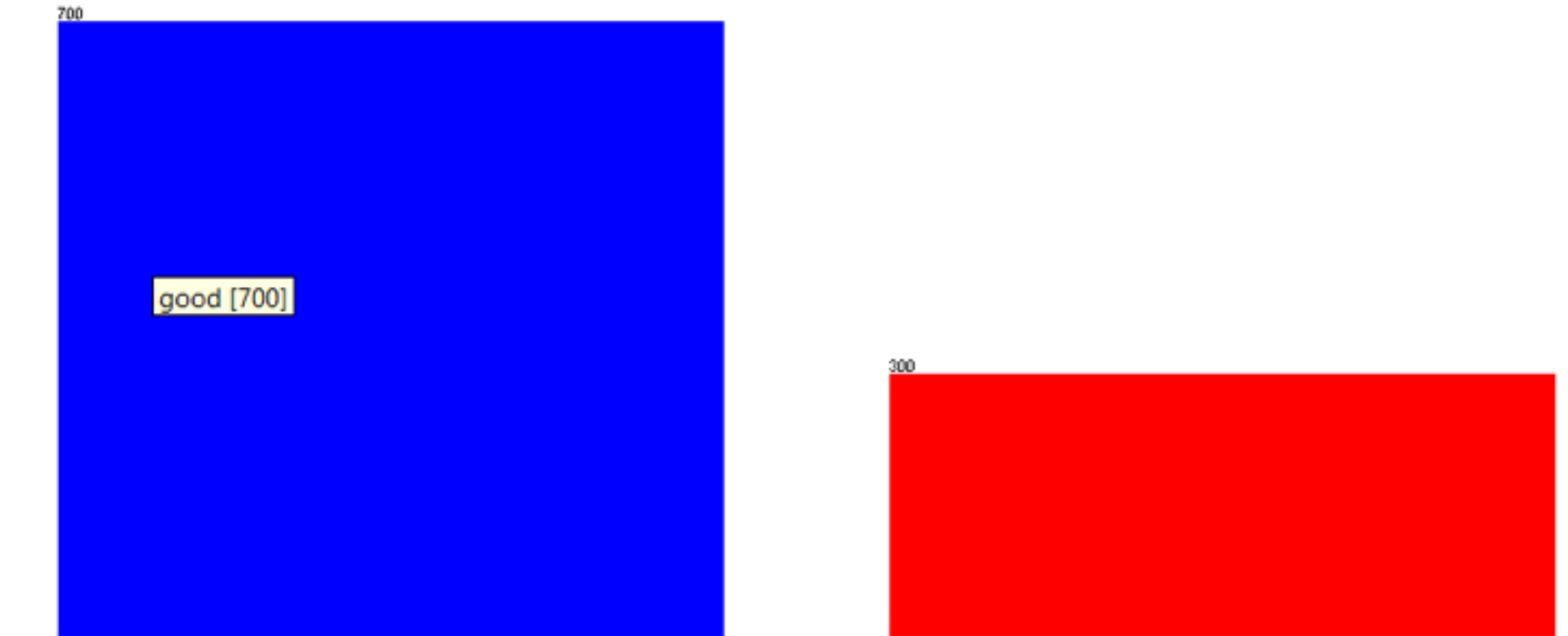
$$N = \frac{\mu}{\mu + \lambda}$$

Classify as positive if: probability of positive $> \frac{\mu}{\mu + \lambda}$

$$\frac{1}{2} > \frac{2}{3} > \frac{2}{6}$$

Cost-sensitive classification

Credit dataset.



✓ Cost-sensitive classification with Weka

- Naïve Bayes.

=== Summary ===

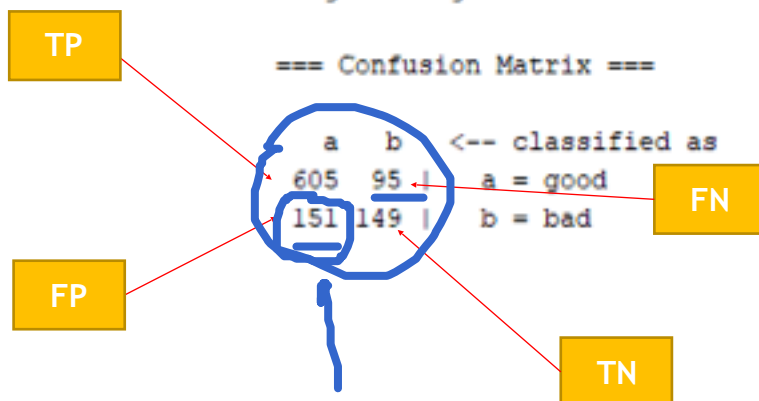
Correctly Classified Instances	754	75.4	%
Incorrectly Classified Instances	246	24.6	%
Kappa statistic	0.3813		
Mean absolute error	0.2936		
Root mean squared error	0.4201		
Relative absolute error	69.8801	%	
Root relative squared error	91.6718	%	
Total Number of Instances	1000		

=== Detailed Accuracy By Class ===

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
	0.864	0.503	0.800	0.864	0.831	0.385	0.787	0.891	good
	0.497	0.136	0.611	0.497	0.548	0.385	0.787	0.577	bad
Weighted Avg.	0.754	0.393	0.743	0.754	0.746	0.385	0.787	0.797	

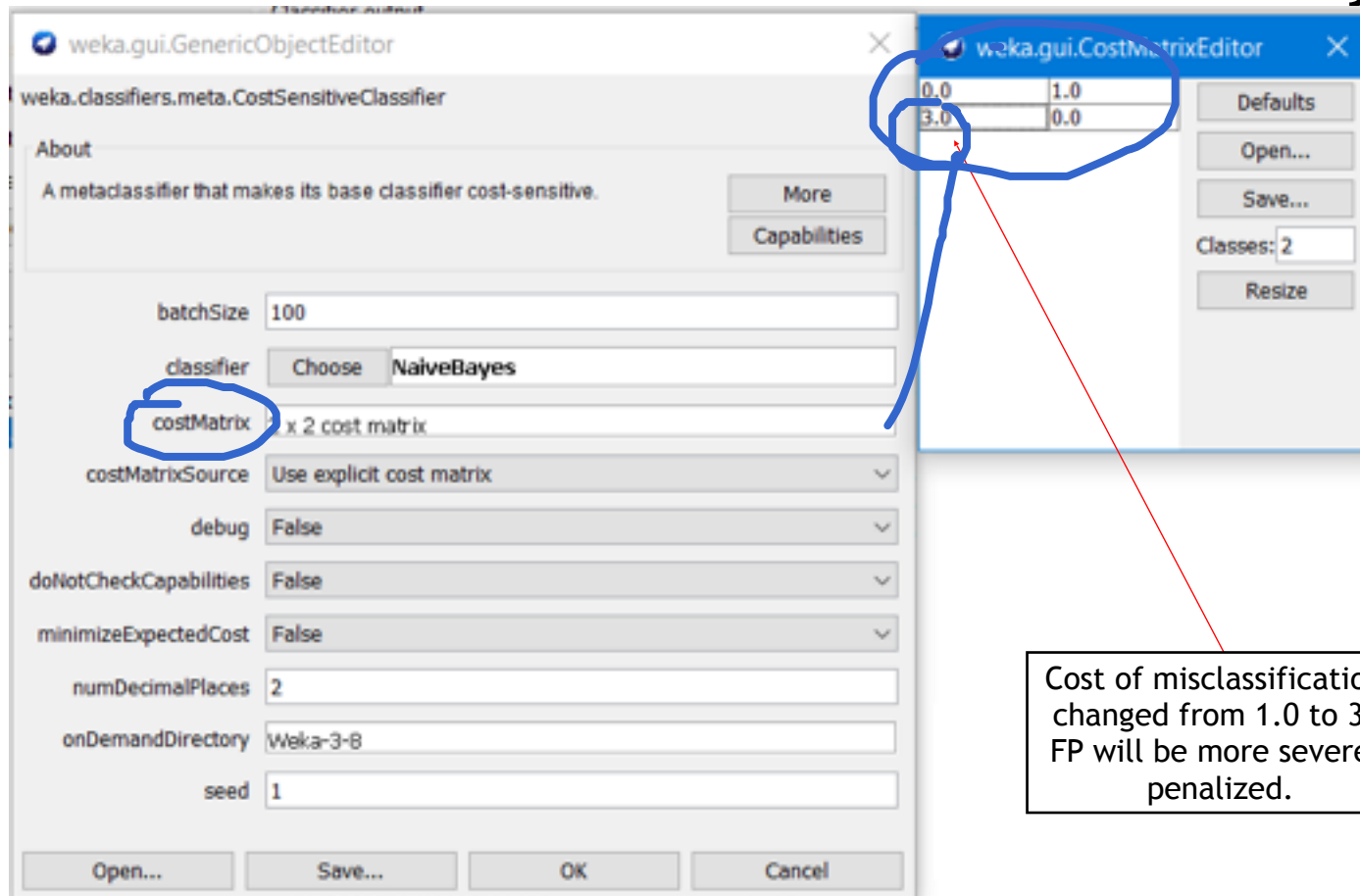
=== Confusion Matrix ===

a	b	<-- classified as
605	95	a = good
151	149	b = bad



Cost-sensitive classification with Weka

- Cost-sensitive classifier with Naïve Bayes.



$$\frac{\mu}{\mu + \lambda} = \frac{3}{3 + 1} = 0.75$$

Cost-sensitive classification with Weka

- Cost-sensitive classifier with Naïve Bayes.

=== Summary ===

Correctly Classified Instances	720	72	%
Incorrectly Classified Instances	280	28	%
Kappa statistic	0.4078		
Mean absolute error	0.3367		
Root mean squared error	0.4448		
Relative absolute error	80.1422	%	
Root relative squared error	97.068	%	
Total Number of Instances	1000		

=== Detailed Accuracy By Class ===

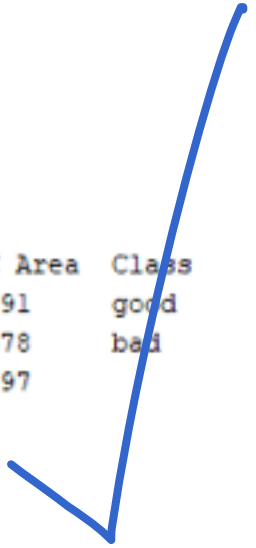
	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
	0.706	0.247	0.870	0.706	0.779	0.425	0.787	0.891	good
	0.753	0.294	0.523	0.753	0.617	0.425	0.787	0.578	bad
Weighted Avg.	0.720	0.261	0.766	0.720	0.731	0.425	0.787	0.797	

=== Confusion Matrix ===

a b <-- classified as
494 206 | a = good
74 226 | b = bad

But also TP
reduced

FPs
reduced



Assign class weights

- In Keras you can assign class weights.

`fit()` function in keras has a `class_weight` parameter (see <https://keras.io/models/model/>).

```
class_weight = {0: 1.,  
                1: 9.}
```

Optional dictionary mapping class indices (integers) to a weight (float) value, used for weighting the loss function (during training only). This can be useful to tell the model to "pay more attention" to samples from an under-represented class.

`max_v` = number of points with majority class

`weight_minority` = `max_v` / (# of minority points)

`weight_majority` = `max_v` / `max_v` = 1

This is just one way to compute class weights

Tools

- Python imbalanced-learn library: <https://github.com/scikit-learn-contrib/imbalanced-learn>
- Weka also has oversampling methods and a cost sensitive meta classifier: <https://weka.wikispaces.com/CostSensitiveClassifier>

Performance metrics

- Most of the time accuracy will not be enough to assess performance.



- $accuracy = \frac{TP+TN}{P+N}$

Percentage of correctly classified instances.

- $sensitivity = \frac{TP}{P}$

The proportion of positives that are correctly identified as such.

- $precision = \frac{TP}{TP+FP}$

Equivalently, it is the fraction of relevant instances among the selected ones.

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Matthews correlation coefficient (takes into account imba

Supporting materials: imbalanced data

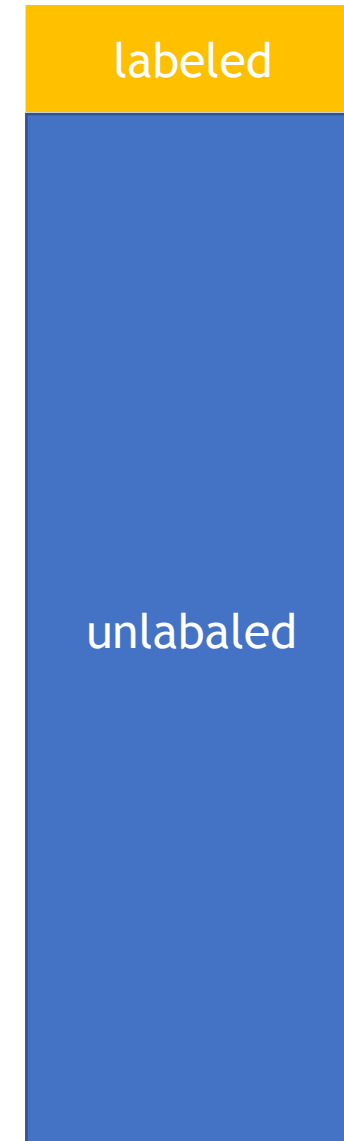
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16, 321-357.
- Kotsiantis, S., Kanellopoulos, D., & Pintelas, P. (2006). Handling imbalanced datasets: A review. *GESTS International Transactions on Computer Science and Engineering*, 30(1), 25-36.
- Python imbalanced-learn library: <https://github.com/scikit-learn-contrib/imbalanced-learn>
- Weka also has oversampling methods and a cost sensitive meta classifier: <https://weka.wikispaces.com/CostSensitiveClassifier>
- Cost-sensitive classification video: <https://www.youtube.com/watch?v=l9muPldOG30>
- Performance metrics: https://en.wikipedia.org/wiki/Sensitivity_and_specificity



Semi-supervised learning (SSL)

Suitable when just a small proportion of the training data is labeled.

These algorithms try to learn also from the unlabeled data.

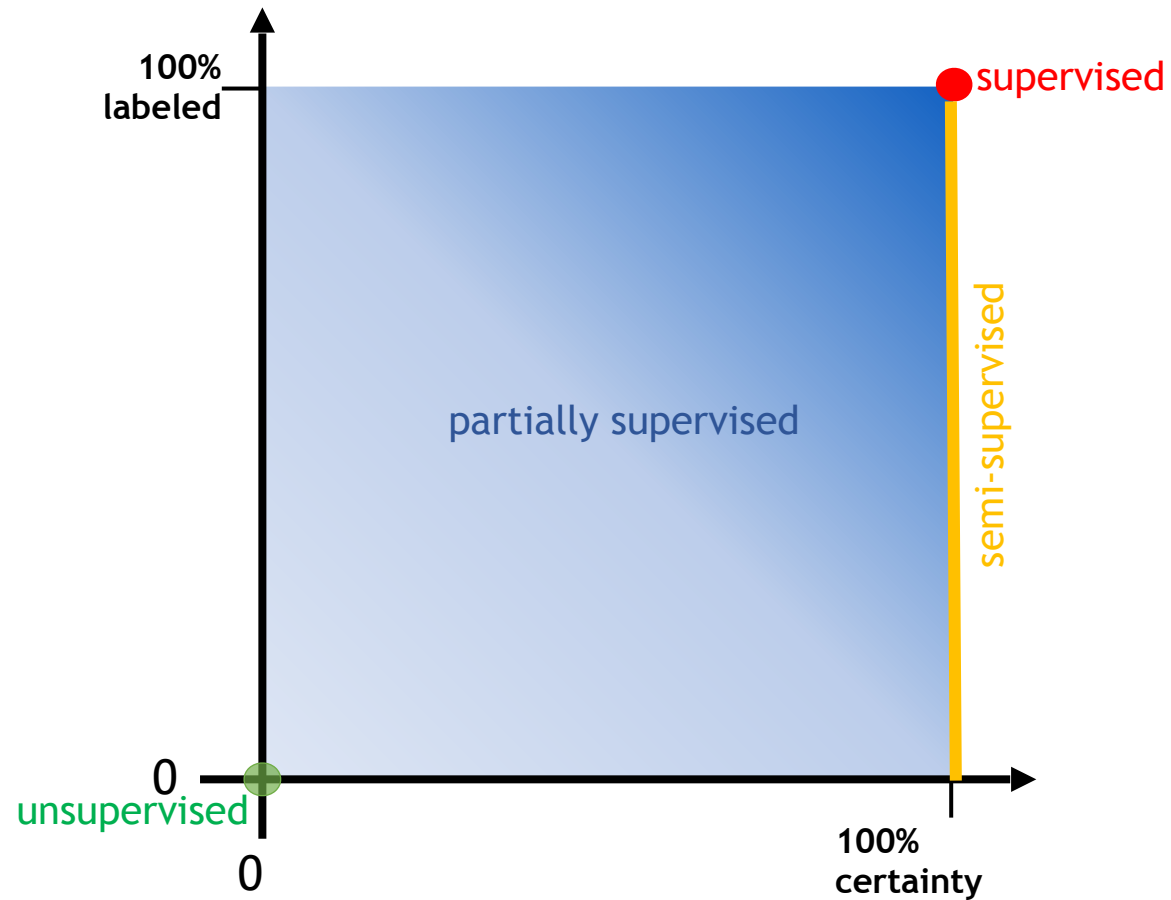


Semi-supervised learning (SSL)

Suitable for datasets with:

- Small amounts of **labeled** data
- Large amounts of **unlabeled** data (labeling requires effort)
- High certainty in labels

Semi-supervised learning (SSL)



Semi-supervised learning (SSL)

- Traditional supervised learning is limited to using labeled data.
- SSL also uses unlabeled data to learn.

Let $(\mathbf{x}, y)$ be a labeled instance and $(\mathbf{x}, \emptyset)$ be an unlabeled instance.

L : a set of n labeled instances.

U : a set of m unlabeled instances.

$n \ll m$

SSL tries to use $L \cup U$ to learn a predictive model.

SSL: Self-learning

Self-learning: a model uses the unlabeled data to improve itself.

L : labeled instances
 U : unlabeled instances

Procedure:

1. Train a model f with L
2. $y' = f(x)$ where $x \in U$
3. $L = L \cup (x, y')$
4. Repeat

Self-learning 'improvements'

- Just add instances with the most confident predictions.
- Perform the procedure with batches of instances instead of one instance at a time.
- Re-assess previous predictions.

SSL is not always
guaranteed to work!
Performance may also
degrade due to noisy
instances.

Remember: garbage in,
garbage out!

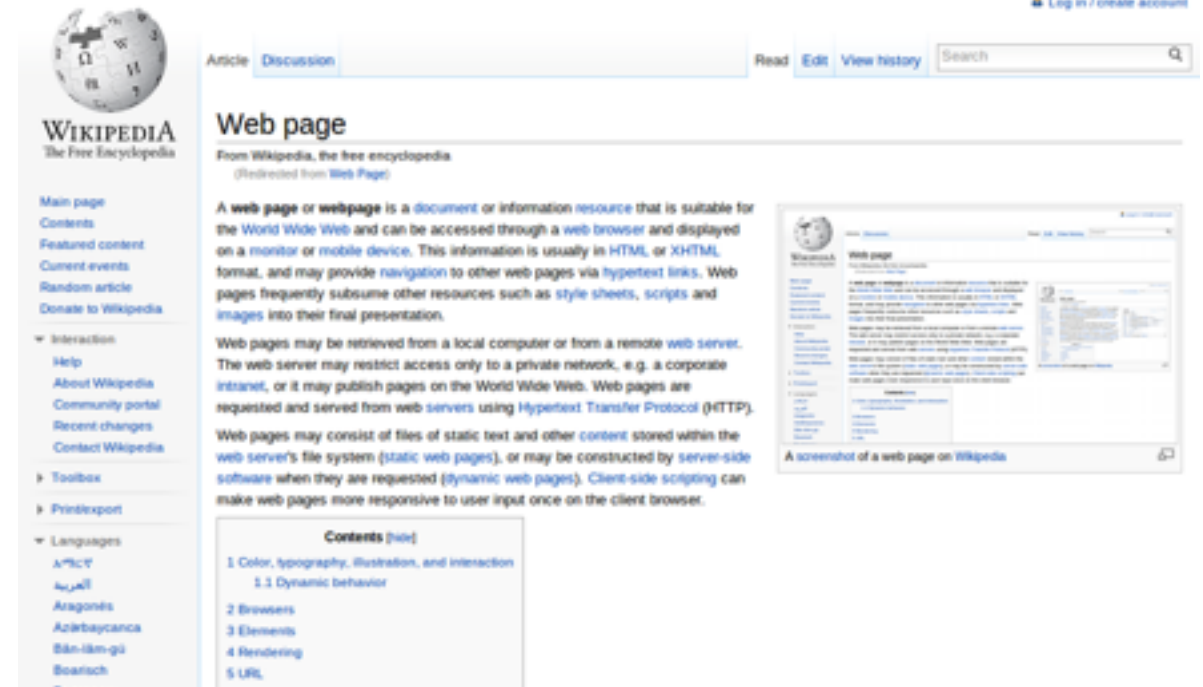


Multi-view learning

Sometimes, an observation can be represented by two independent sets of features or ‘views’.

For example a webpage can be characterized by its content but also by the links’ text pointing to it.

This view redundancy can be used for semi-supervised learning!



Multi-view learning

- Conventional algorithms ‘concatenate’ all views.
- This approach might cause overfitting with small training sets.
- Not physically meaningful since each view has specific statistical properties.

Multi-view learning

Multi-view learning takes advantage of all views to jointly optimize and exploit the redundant views of the same input data to improve performance.

Multi-view learning

Co-Training (Blum, A., & Mitchell, T.) is a type of semi-supervised algorithm.

Two classifiers work together to enlarge the training set L and increase performance.



Co-training algorithm

Given:

- a set L of labeled training examples
- a set U of unlabeled examples

Create a pool U' of examples by choosing u examples at random from U

Loop for k iterations:

Some implementations use independent L for each view.

Use L to train a classifier h_1 that considers only the x_1 portion of x

Use L to train a classifier h_2 that considers only the x_2 portion of x

Allow h_1 to label p positive and n negative examples from U'

Allow h_2 to label p positive and n negative examples from U'

Add these self-labeled examples to L

Randomly choose $2p + 2n$ examples from U to replenish U'

Co-training algorithm

Assumptions

- A feature split into two views exists.
- Each feature split (view) is sufficient to train a good classifier.
- The views are conditionally independent given the class.

Co-training algorithm

How to combine the results?

- Multiply output probabilities.
- Choose the class with maximum probability among the two models.
- Train a single model after the last iteration.

Tri-training

Tri-training (Zhou and Li)

- Use three learners. If two agree, the data is used to teach the third learner.

From Co-training to tri-training

One sees light,
and the other one sees dark
but they always work together
to enlarge the training set.

Sometimes they disagree,
so they keep independent labeled sets
but at the end, they always become friends.

Sometimes they make mistakes
and they will propagate
causing the overall performance to degrade.

Fortunately, they met a new learner
and whenever they agree,
the data point is shared with it.

Now the three train together
predict together
and will live forever.



Supporting materials: SSL, multi-view learning and co-training.

- Blum, A., & Mitchell, T. (1998, July). Combining labeled and unlabeled data with co-training. In *Proceedings of the eleventh annual conference on Computational learning theory* (pp. 92-100). ACM.
- Zhou, Z. H., & Li, M. (2005). Tri-training: Exploiting unlabeled data using three classifiers. *IEEE Transactions on knowledge and Data Engineering*, 17(11), 1529-1541.
- Chapelle, O., Scholkopf, B., & Zien, A. (2009). Semi-supervised learning (chapelle, o. et al., eds.; 2006)[book reviews]. *IEEE Transactions on Neural Networks*, 20(3), 542-542.